

Entrega Final - Proyecto de Ciencia de Datos

Contexto del Proyecto

En este proyecto actué como Científico de Datos en una empresa de productos de consumo masivo. El objetivo principal fue analizar el comportamiento histórico de las ventas y construir un modelo predictivo que permitiera estimar la demanda futura de productos clave como Vanish y Lysol.

Para lograrlo, trabajé con un modelo dimensional de datos (tablas de dimensión y tabla de hechos), realizando carga, limpieza, consolidación, análisis exploratorio, segmentación mediante clustering K-Means y modelado de series temporales con ARIMA.

Contexto del Negocio

Este proyecto simula el rol de un Científico de Datos dentro de una empresa de consumo masivo. El objetivo es transformar datos históricos de ventas en insights accionables para optimizar inventarios, planificación de demanda y decisiones estratégicas.

Preguntas clave de negocio:

- ¿Qué productos generan mayor valor?
- ¿Cómo evolucionan las ventas en el tiempo y por región?
- ¿Qué segmentos presentan mejor rendimiento?
- ¿Cómo predecir la demanda futura?

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# ===== CARGA DE DATOS =====
print( Cargando archivos del proyecto...)
```

```
df_sales = pd.read_csv('FACT_SALES.csv')
df_product = pd.read_csv('DIM_PRODUCT.csv')
df_category = pd.read_csv('DIM_CATEGORY.csv')
df_segment = pd.read_csv('DIM_SEGMENT.csv')
df_calendar = pd.read_csv('DIM_CALENDAR.csv')

print("✅ Archivos cargados correctamente!\n")
print(f"FACT_SALES    → {df_sales.shape}")
print(f"DIM_PRODUCT    → {df_product.shape}")
print(f"DIM_CATEGORY   → {df_category.shape}")
print(f"DIM_SEGMENT    → {df_segment.shape}")
print(f"DIM_CALENDAR   → {df_calendar.shape}")

print("\n" + "="*60)
print("INFO FACT_SALES:")
print(df_sales.info())

print("\nPrimeras 5 filas de Ventas:")
print(df_sales.head())
```

 Cargando archivos del proyecto...

✅ Archivos cargados correctamente!

```
FACT_SALES      → (122002, 6)
DIM_PRODUCT     → (505, 9)
DIM_CATEGORY    → (5, 2)
DIM_SEGMENT     → (53, 6)
DIM_CALENDAR    → (156, 5)
```

```
=====
INFO FACT_SALES:
<class 'pandas.DataFrame'>
RangeIndex: 122002 entries, 0 to 122001
Data columns (total 6 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   WEEK                                  122002 non-null  str
 1   ITEM_CODE                            122002 non-null  str
 2   TOTAL_UNIT_SALES                     122002 non-null  float64
 3   TOTAL_VALUE_SALES                    122002 non-null  float64
 4   TOTAL_UNIT_AVG_WEEKLY_SALES           122002 non-null  float64
 5   REGION                               122002 non-null  str
dtypes: float64(3), str(3)
memory usage: 5.6 MB
None
```

Primeras 5 filas de Ventas:

	WEEK	ITEM_CODE	TOTAL_UNIT_SALES	TOTAL_VALUE_SALES \
0	34-22	7501058792808BP2	0.006	0.139
1	34-22	7501058715883	0.487	116.519
2	34-22	7702626213774	1.391	68.453
3	34-22	7501058716422	0.022	1.481
4	34-22	7501058784353	2.037	182.839

	TOTAL_UNIT_AVG_WEEKLY_SALES	REGION
0	1.000	TOTAL AUTOS AREA 5
1	2.916	TOTAL AUTOS AREA 5
2	5.171	TOTAL AUTOS AREA 5
3	1.833	TOTAL AUTOS AREA 5
4	5.375	TOTAL AUTOS AREA 5

Herramientas Utilizadas

En este proyecto utilicé:

- **Python** (Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn)
- **Statsmodels** para modelado de series temporales
- **SQL Server** para consultas relacionales
- **Power BI** para visualización interactiva

```
In [2]: calendar = pd.read_csv("DIM_CALENDAR.csv")
product = pd.read_csv("DIM_PRODUCT.csv")
sales = pd.read_csv("FACT_SALES.csv")

print("Datos cargados correctamente")
```

Datos cargados correctamente

Exploración Inicial de los Datos

Revisé la estructura y calidad de las tablas para entender su organización y detectar posibles problemas antes de la consolidación.

```
In [3]: print("SHAPES:")
print(calendar.shape, product.shape, sales.shape)
print("\nINFO SALES:")
sales.info()
```

SHAPES:

(156, 5) (505, 9) (122002, 6)

INFO SALES:

<class 'pandas.DataFrame'>

RangeIndex: 122002 entries, 0 to 122001

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	WEEK	122002 non-null	str
1	ITEM_CODE	122002 non-null	str
2	TOTAL_UNIT_SALES	122002 non-null	float64
3	TOTAL_VALUE_SALES	122002 non-null	float64
4	TOTAL_UNIT_AVG_WEEKLY_SALES	122002 non-null	float64
5	REGION	122002 non-null	str

dtypes: float64(3), str(3)

memory usage: 5.6 MB

Limpieza y Preparación de Datos

Realicé la limpieza y normalización de las llaves para garantizar consistencia en las uniones posteriores.

In [4]: `print(df_product.head())`

	MANUFACTURER	BRAND	ITEM \
0	INDS. ALEN	CLORALEX	0000075000592
1	INDS. ALEN	CLORALEX	0000075000608
2	INDS. ALEN	CLORALEX	0000075000615
3	INDS. ALEN	CLORALEX	0000075000622
4	INDS. ALEN	CLORALEX	0000075000639

		ITEM_DESCRIPTION	CATEGORY	FORMAT \
0	CLORALEX EL RENDIDOR	BOT.PLAST. 250ML NAL. 000...	1	LIQUIDO
1	CLORALEX EL RENDIDOR	BOT.PLAST. 500ML NAL. 000...	1	LIQUIDO
2	CLORALEX EL RENDIDOR	BOT.PLAST. 950ML NAL. 000...	1	LIQUIDO
3	CLORALEX EL RENDIDOR	BOT.PLAST. 2000ML NAL 000...	1	LIQUIDO
4	CLORALEX EL RENDIDOR	BOT.PLAST. 3750ML NAL 000...	1	LIQUIDO

	ATTR1	ATTR2	ATTR3
0	CLORO	CLORO	NO DEFINIDO
1	CLORO	CLORO	NO DEFINIDO
2	CLORO	CLORO	NO DEFINIDO
3	CLORO	CLORO	NO DEFINIDO
4	CLORO	CLORO	NO DEFINIDO

```
In [5]: print(df_product.columns)
```

```
Index(['MANUFACTURER', 'BRAND', 'ITEM', 'ITEM_DESCRIPTION', 'CATEGORY',
      'FORMAT', 'ATTR1', 'ATTR2', 'ATTR3'],
      dtype='str')
```

```
In [6]: df_product.columns = df_product.columns.str.strip().str.upper()
print(df_product.columns)
```

```
Index(['MANUFACTURER', 'BRAND', 'ITEM', 'ITEM_DESCRIPTION', 'CATEGORY',
      'FORMAT', 'ATTR1', 'ATTR2', 'ATTR3'],
      dtype='str')
```

```
In [7]: # ===== NORMALIZACIÓN DE COLUMNAS =====
for df_name, df in [("Sales", df_sales), ("Product", df_product), ("Category", df_category),
                    ("Segment", df_segment), ("Calendar", df_calendar)]:
    df.columns = df.columns.str.strip().str.upper()
    print(f"✅ {df_name} columns normalized")

print("\nColumnas finales:")
print("Sales:", df_sales.columns.tolist())
print("Product:", df_product.columns.tolist())
```

```

# ===== LIMPIEZA DE LLAVES =====
df_sales["ITEM_CLEAN"] = df_sales["ITEM_CODE"].astype(str).str.extract(r'(\d+)')
df_sales["ITEM_CLEAN"] = df_sales["ITEM_CLEAN"].str.zfill(13)

df_product["ITEM_CLEAN"] = df_product["ITEM"].astype(str).str.extract(r'(\d+)')
df_product["ITEM_CLEAN"] = df_product["ITEM_CLEAN"].str.zfill(13)

print("\n✅ Llaves limpiadas correctamente")
print("Ejemplo ITEM_CLEAN Sales:", df_sales["ITEM_CLEAN"].head())
print("Ejemplo ITEM_CLEAN Product:", df_product["ITEM_CLEAN"].head())

# ===== CONSOLIDACIÓN (MERGE) =====
df_merged = df_sales.merge(df_product, left_on="ITEM_CLEAN", right_on="ITEM_CLEAN", how="left")

df_calendar["WEEK"] = df_calendar["WEEK"].astype(str)
df_merged = df_merged.merge(df_calendar, on="WEEK", how="left")

print("\n" + "="*70)
print("✅ DATASET CONSOLIDADO")
print("Shape final:", df_merged.shape)
print("\nValores nulos después del merge:")
print(df_merged.isnull().sum())

print("\nPrimeras 5 filas:")
print(df_merged.head())

```

- ✅ Sales columns normalized
- ✅ Product columns normalized
- ✅ Category columns normalized
- ✅ Segment columns normalized
- ✅ Calendar columns normalized

Columnas finales:

Sales: ['WEEK', 'ITEM_CODE', 'TOTAL_UNIT_SALES', 'TOTAL_VALUE_SALES', 'TOTAL_UNIT_AVG_WEEKLY_SALES', 'REGION']

Product: ['MANUFACTURER', 'BRAND', 'ITEM', 'ITEM_DESCRIPTION', 'CATEGORY', 'FORMAT', 'ATTR1', 'ATTR2', 'ATTR3']

✓ Llaves limpiadas correctamente

Ejemplo ITEM_CLEAN Sales: 0 7501058792808

1 7501058715883

2 7702626213774

3 7501058716422

4 7501058784353

Name: ITEM_CLEAN, dtype: str

Ejemplo ITEM_CLEAN Product: 0 0000075000592

1 0000075000608

2 0000075000615

3 0000075000622

4 0000075000639

Name: ITEM_CLEAN, dtype: str

=====

✓ DATASET CONSOLIDADO

Shape final: (130338, 20)

Valores nulos después del merge:

```

WEEK
ITEM_CODE
TOTAL_UNIT_SALES
TOTAL_VALUE_SALES
TOTAL_UNIT_AVG_WEEKLY_SALES
REGION
ITEM_CLEAN
MANUFACTURER
BRAND
ITEM
ITEM_DESCRIPTION
CATEGORY
FORMAT
ATTR1
ATTR2
ATTR3
YEAR
MONTH
WEEK_NUMBER
DATE

```

dtype: int64

Primeras 5 filas:

	WEEK	ITEM_CODE	TOTAL_UNIT_SALES	TOTAL_VALUE_SALES \
0	34-22	7501058792808BP2	0.006	0.139
1	34-22	7501058792808BP2	0.006	0.139
2	34-22	7501058715883	0.487	116.519
3	34-22	7702626213774	1.391	68.453
4	34-22	7501058716422	0.022	1.481

	TOTAL_UNIT_AVG_WEEKLY_SALES	REGION	ITEM_CLEAN \
0	1.000	TOTAL AUTOS AREA 5	7501058792808
1	1.000	TOTAL AUTOS AREA 5	7501058792808
2	2.916	TOTAL AUTOS AREA 5	7501058715883
3	5.171	TOTAL AUTOS AREA 5	7702626213774
4	1.833	TOTAL AUTOS AREA 5	7501058716422

	MANUFACTURER	BRAND	ITEM \
0	RECKITT	VANISH	7501058792808BP1
1	RECKITT	VANISH	7501058792808BP2
2	RECKITT	VANISH	7501058715883
3	RECKITT	VANISH	7702626213774

4 RECKITT VANISH 7501058716422

	ITEM_DESCRIPTION	CATEGORY	FORMAT	\
0	VANISHOXIACTIONCRYSTALWHITEBOTE900GR+MMROSADOY...	1	POLVO	
1	VANISHOXIACTIONROSADOYPACK120GR+MMCRYSTALWHITE...	1	POLVO	
2	VANISH OXI ACTION GOLD QUITAMANCHAS BOLSA 1.8K...	1	POLVO	
3	VANISH OXI ACTION ROSA QUITAMANCHAS DOYPACK 24...	1	POLVO	
4	VANISH OXI ACTION GOLD QUITAMANCHA AHORRO DEL ...	1	POLVO	

	ATTR1	ATTR2	ATTR3	YEAR	MONTH	WEEK_NUMBER	DATE
0	SAFE BLEACH	FABRIC TREATMENT	BLANCO	2022	8	34	2022-08-28
1	SAFE BLEACH	FABRIC TREATMENT	ROSA	2022	8	34	2022-08-28
2	SAFE BLEACH	FABRIC TREATMENT	ROSA	2022	8	34	2022-08-28
3	SAFE BLEACH	FABRIC TREATMENT	ROSA	2022	8	34	2022-08-28
4	SAFE BLEACH	FABRIC TREATMENT	ROSA	2022	8	34	2022-08-28

Consolidación del Dataset

Integré las tablas dimensionales con la tabla de hechos para crear un dataset unificado.

```
In [8]: # ===== MERGE FINAL =====
print("🚀 Iniciando consolidación...")

sales_product = df_sales.merge(df_product, left_on="ITEM_CLEAN", right_on="ITEM_CLEAN", how="left")

print("✅ Merge Sales + Product → Shape:", sales_product.shape)

df_calendar["WEEK"] = df_calendar["WEEK"].astype(str).str.strip()

df = sales_product.merge(df_calendar, on="WEEK", how="left")

print("\n🎉 ¡DATASET CONSOLIDADO EXITOSAMENTE!")
print("Shape final del dataset:", df.shape)
print("Columnas finales:", df.columns.tolist())

print("\nNulos en el dataset final:")
print(df.isnull().sum())

df.to_csv("dataset_consolidado.csv", index=False)
print("\n✅ Archivo guardado como: dataset_consolidado.csv")
```

 Iniciando consolidación...

 Merge Sales + Product → Shape: (130338, 16)

 ¡DATASET CONSOLIDADO EXITOSAMENTE!

Shape final del dataset: (130338, 20)

Columnas finales: ['WEEK', 'ITEM_CODE', 'TOTAL_UNIT_SALES', 'TOTAL_VALUE_SALES', 'TOTAL_UNIT_AVG_WEEKLY_SALES', 'REGION', 'ITEM_CLEAN', 'MANUFACTURER', 'BRAND', 'ITEM', 'ITEM_DESCRIPTION', 'CATEGORY', 'FORMAT', 'ATTR1', 'ATTR2', 'ATTR3', 'YEAR', 'MONTH', 'WEEK_NUMBER', 'DATE']

Nulos en el dataset final:

WEEK	0
ITEM_CODE	0
TOTAL_UNIT_SALES	0
TOTAL_VALUE_SALES	0
TOTAL_UNIT_AVG_WEEKLY_SALES	0
REGION	0
ITEM_CLEAN	0
MANUFACTURER	0
BRAND	0
ITEM	0
ITEM_DESCRIPTION	0
CATEGORY	0
FORMAT	0
ATTR1	0
ATTR2	0
ATTR3	0
YEAR	0
MONTH	0
WEEK_NUMBER	0
DATE	0

dtype: int64

 Archivo guardado como: dataset_consolidado.csv

Tratamiento de Nulos y Duplicados

```
In [9]: # Tratamiento de duplicados y nulos
print("Duplicados antes de eliminar:", df.duplicated().sum())
df = df.drop_duplicates()

print("\nNulos por columna:")
```

```

print(df.isnull().sum())

# Imputación segura (mejorada)
numeric_cols = df.select_dtypes(include=np.number).columns
cat_cols = df.select_dtypes(include='object').columns

df[numeric_cols] = df[numeric_cols].fillna(0)
df[cat_cols] = df[cat_cols].fillna("Unknown")
print("✅ Nulos imputados de forma segura y duplicados eliminados")

```

Duplicados antes de eliminar: 0

Nulos por columna:

WEEK	0
ITEM_CODE	0
TOTAL_UNIT_SALES	0
TOTAL_VALUE_SALES	0
TOTAL_UNIT_AVG_WEEKLY_SALES	0
REGION	0
ITEM_CLEAN	0
MANUFACTURER	0
BRAND	0
ITEM	0
ITEM_DESCRIPTION	0
CATEGORY	0
FORMAT	0
ATTR1	0
ATTR2	0
ATTR3	0
YEAR	0
MONTH	0
WEEK_NUMBER	0
DATE	0

dtype: int64

✅ Nulos imputados de forma segura y duplicados eliminados

EDA por Región y Categoría

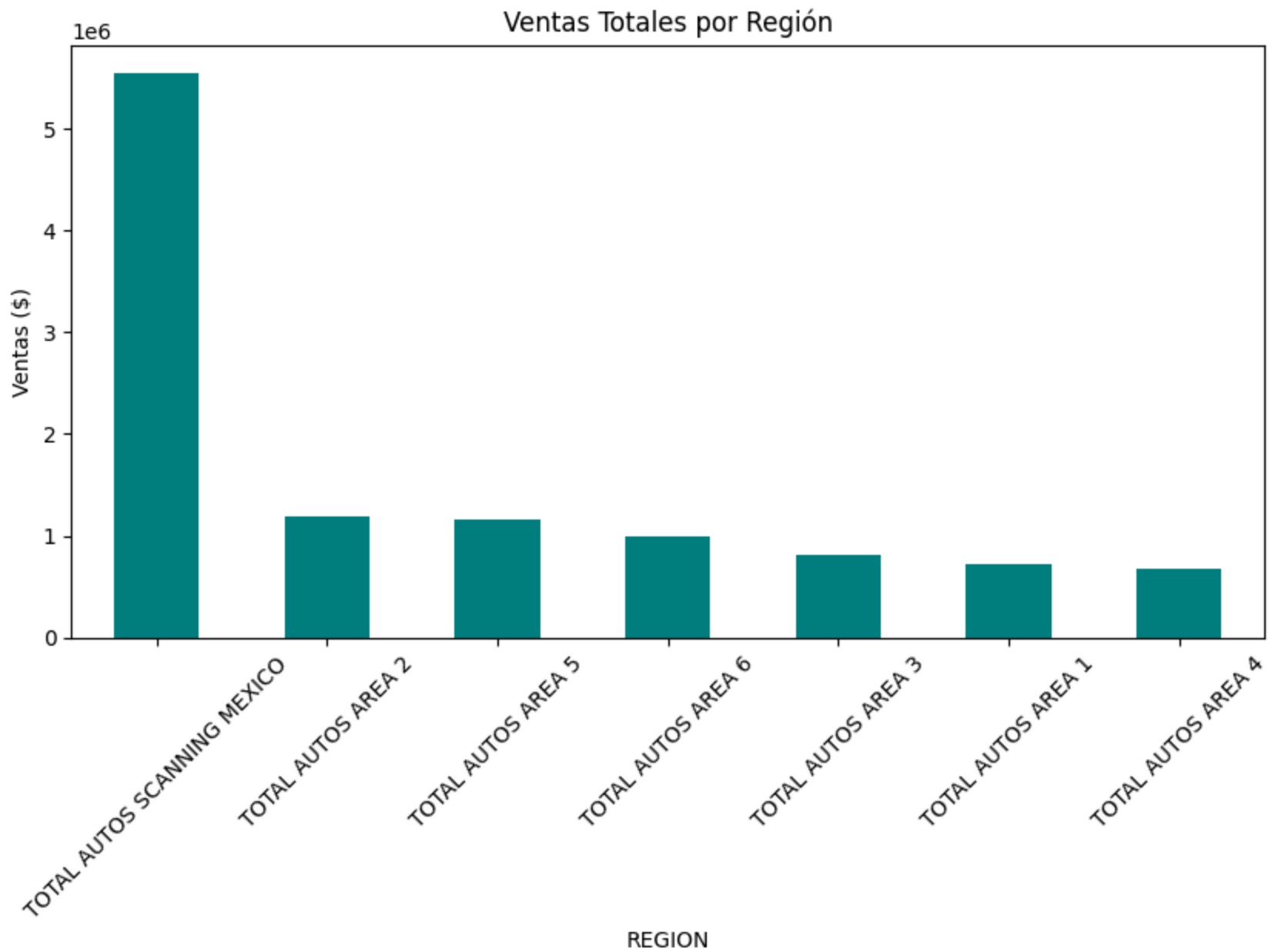
```

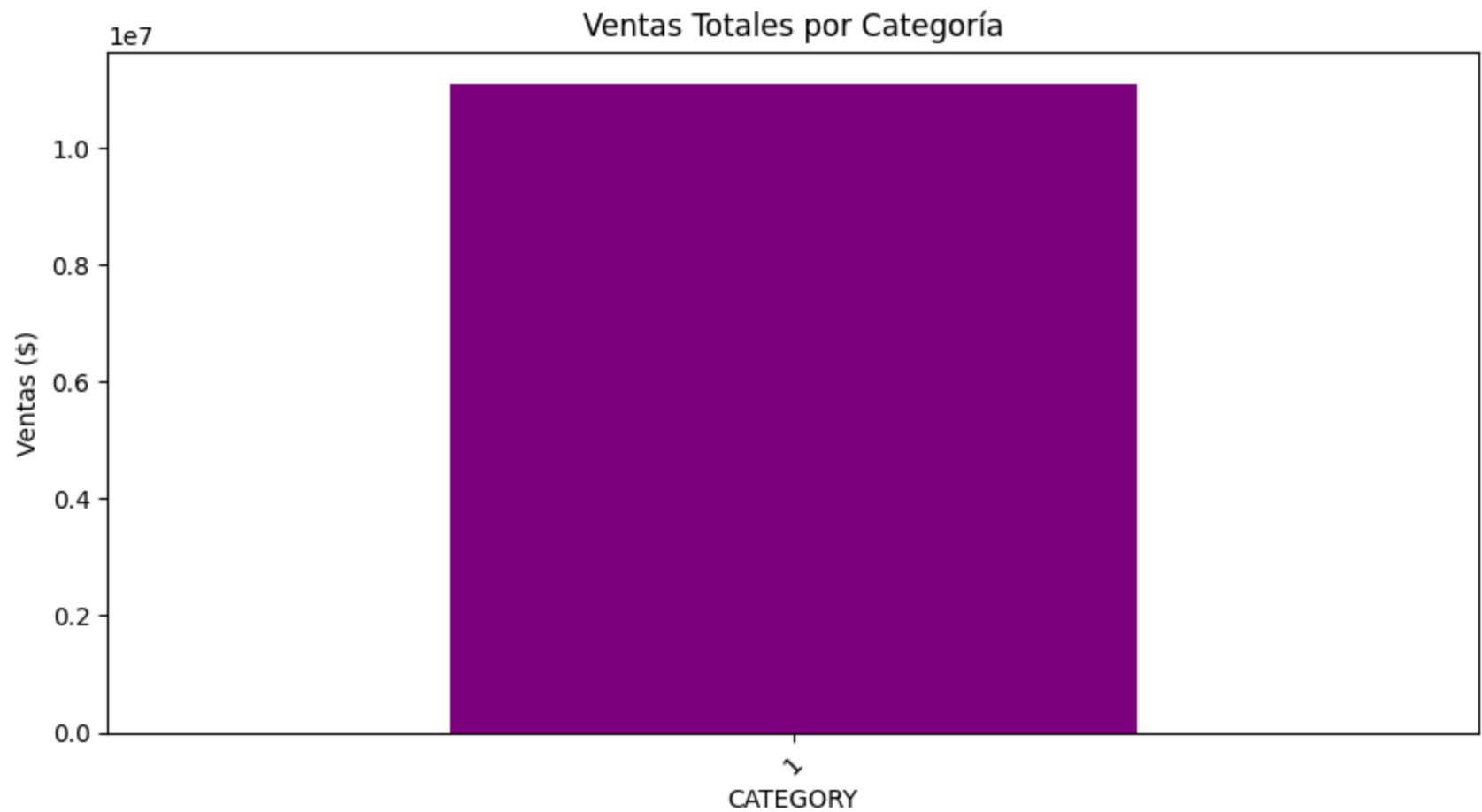
In [10]: plt.figure(figsize=(10,5))
df.groupby("REGION")["TOTAL_VALUE_SALES"].sum().sort_values(ascending=False).plot(kind="bar", color='teal')
plt.title("Ventas Totales por Región")
plt.ylabel("Ventas ($)")

```

```
plt.xticks(rotation=45)
plt.show()

plt.figure(figsize=(10,5))
df.groupby("CATEGORY")["TOTAL_VALUE_SALES"].sum().sort_values(ascending=False).plot(kind="bar", color='purple')
plt.title("Ventas Totales por Categoría")
plt.ylabel("Ventas ($)")
plt.xticks(rotation=45)
plt.show()
```





Construcción de la Serie Temporal

```
In [11]: # Convertir DATE a datetime
df["DATE"] = pd.to_datetime(df["DATE"])

print("🔍 Buscando productos Vanish y Lysol...")

mask = (
    df['BRAND'].astype(str).str.upper().str.contains('VANISH|LYSOL', na=False) |
    df['ITEM_DESCRIPTION'].astype(str).str.upper().str.contains('VANISH|LYSOL', na=False)
)
```

```

forecast_df = df[mask].copy()

print(f"✅ Registros encontrados de Vanish/Lysol: {len(forecast_df)}")

ts = forecast_df.groupby("DATE")["TOTAL_VALUE_SALES"].sum().reset_index()
ts = ts.sort_values("DATE")
ts.set_index("DATE", inplace=True)
ts = ts.rename(columns={"TOTAL_VALUE_SALES": "VENTAS"})

print("\nPrimeras filas de la serie temporal:")
print(ts.head())
print("\nShape de la serie temporal:", ts.shape)

```

🔍 Buscando productos Vanish y Lysol...

✅ Registros encontrados de Vanish/Lysol: 37062

Primeras filas de la serie temporal:

DATE	VENTAS
------	--------

2022-01-09	30991.419
2022-01-16	29996.760
2022-01-23	26003.775
2022-01-30	25770.141
2022-02-06	25910.258

Shape de la serie temporal: (80, 1)

Prueba de Estacionaridad

```

In [12]: from statsmodels.tsa.stattools import adfuller

print("📊 Realizando prueba ADF de Estacionaridad...")
result = adfuller(ts["VENTAS"])

print("ADF Statistic: {:.4f}".format(result[0]))
print("p-value: {:.4f}".format(result[1]))

print("\nInterpretación:")
if result[1] < 0.05:
    print("✅ La serie es ESTACIONARIA (bueno para ARIMA)")

```

```
else:
    print("❌ La serie NO es estacionaria. Se recomienda diferenciar.")
```

 Realizando prueba ADF de Estacionaridad...

ADF Statistic: -0.6714

p-value: 0.8540

Interpretación:

❌ La serie NO es estacionaria. Se recomienda diferenciar.

División de Datos para Modelado

```
In [13]: train_size = int(len(ts) * 0.8)
train = ts.iloc[:train_size]
test = ts.iloc[train_size:]

print("Train shape:", train.shape)
print("Test shape:", test.shape)
```

Train shape: (64, 1)

Test shape: (16, 1)

Justificación del Modelo ARIMA

```
In [14]: from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np

print("🔄 Dividiendo datos en Train y Test...")
train_size = int(len(ts) * 0.8)
train = ts.iloc[:train_size]
test = ts.iloc[train_size:]

print("\n🚀 Entrenando modelo ARIMA(1,1,1)...")
model = ARIMA(train["VENTAS"], order=(1,1,1))
model_fit = model.fit()

print("✅ Modelo entrenado correctamente")

pred = model_fit.forecast(steps=len(test))
```

```

y_true = test["VENTAS"].values
y_pred = pred.values

mse = mean_squared_error(y_true, y_pred)
mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mse)

mask = y_true != 0
mape = np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100 if mask.sum() > 0 else np.nan

print("\n" + "="*60)
print("📊 MÉTRICAS FINALES DEL MODELO ARIMA")
print(f"MSE = {mse:,.2f}")
print(f"MAE = {mae:,.2f}")
print(f"RMSE = {rmse:,.2f}")
print(f"MAPE = {mape:,.2f}%")

```

🔄 Dividiendo datos en Train y Test...

🚀 Entrenando modelo ARIMA(1,1,1)...

D:\Documentos\Ebac\Finales\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.

```
self._init_dates(dates, freq)
```

D:\Documentos\Ebac\Finales\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.

```
self._init_dates(dates, freq)
```

D:\Documentos\Ebac\Finales\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:473: ValueWarning: A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.

```
self._init_dates(dates, freq)
```

✅ Modelo entrenado correctamente

=====

📊 MÉTRICAS FINALES DEL MODELO ARIMA

MSE = 19,364,990.67

MAE = 3,136.74

RMSE = 4,400.57

MAPE = 9.14%

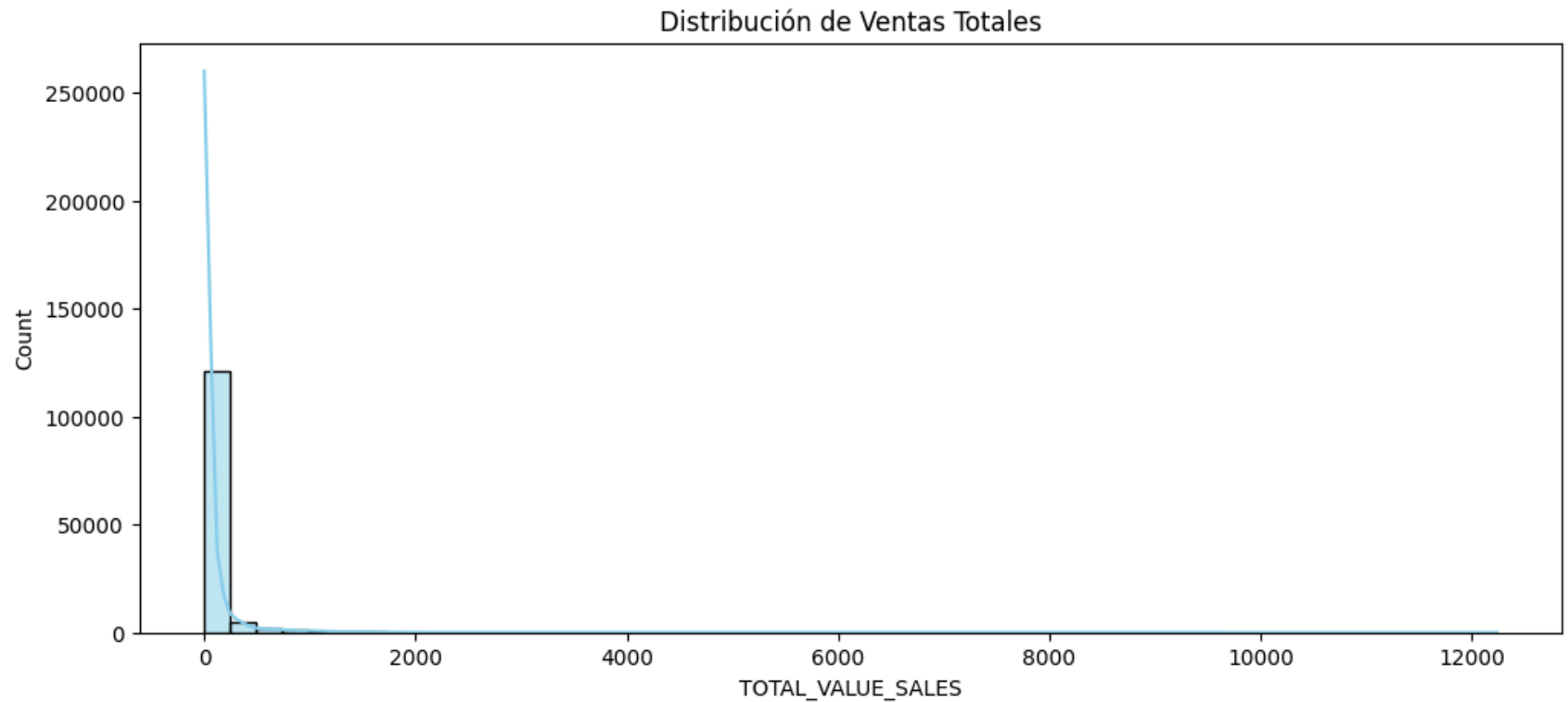
D:\Documentos\Ebac\Finales\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:837: ValueWarning: No supported index is available. Prediction results will be given with an integer index beginning at `start`.

```
return get_prediction_index(
```

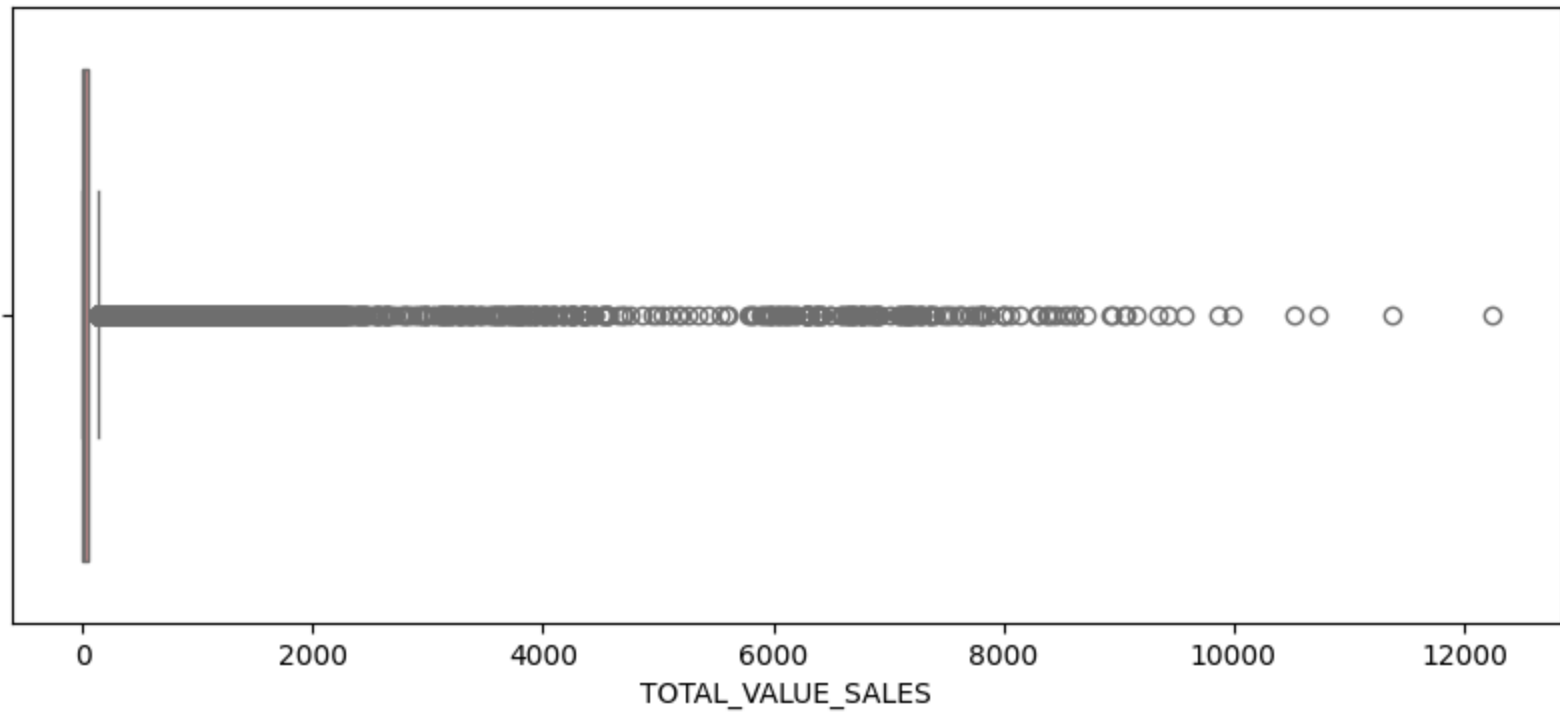
EDA Adicional: Histogramas y Boxplots

```
In [15]: plt.figure(figsize=(12,5))
sns.histplot(df['TOTAL_VALUE_SALES'], bins=50, kde=True, color='skyblue')
plt.title('Distribución de Ventas Totales')
plt.show()

plt.figure(figsize=(10,4))
sns.boxplot(x=df['TOTAL_VALUE_SALES'], color='lightcoral')
plt.title('Boxplot de Ventas - Detección de Outliers')
plt.show()
```



Boxplot de Ventas - Detección de Outliers



Regresión Lineal Múltiple

```
In [16]: print("Columnas disponibles:", df.columns.tolist())

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

# Regresión Lineal Múltiple
features = ["TOTAL_UNIT_SALES"]
if "TOTAL_UNIT_AVG_WEEKLY_SALES" in df.columns:
    features.append("TOTAL_UNIT_AVG_WEEKLY_SALES")

X = df[features]
y = df["TOTAL_VALUE_SALES"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

lr = LinearRegression()
lr.fit(X_train, y_train)
pred_lr = lr.predict(X_test)

print("◆ RESULTADOS REGRESIÓN LINEAL MÚLTIPLE")
print(f"R² Score: {r2_score(y_test, pred_lr):.4f}")
print(f"MAE: {mean_absolute_error(y_test, pred_lr):.2f}")
print(f"MSE: {mean_squared_error(y_test, pred_lr):.2f}")
```

Columnas disponibles: ['WEEK', 'ITEM_CODE', 'TOTAL_UNIT_SALES', 'TOTAL_VALUE_SALES', 'TOTAL_UNIT_AVG_WEEKLY_SALES', 'REGION', 'ITEM_CLEAN', 'MANUFACTURER', 'BRAND', 'ITEM', 'ITEM_DESCRIPTION', 'CATEGORY', 'FORMAT', 'ATTR1', 'ATTR2', 'ATTR3', 'YEAR', 'MONTH', 'WEEK_NUMBER', 'DATE']

◆ RESULTADOS REGRESIÓN LINEAL MÚLTIPLE

R² Score: 0.8094

MAE: 41.64

MSE: 19,754.07

Evaluación del Modelo con AIC y BIC

```
In [17]: print("AIC:", model_fit.aic)
         print("BIC:", model_fit.bic)
```

AIC: 1192.447451681128

BIC: 1198.8768558603026

Predicción y Evaluación del Modelo

```
In [18]: import matplotlib.pyplot as plt
         import seaborn as sns

         pred = model_fit.forecast(steps=len(test))
         residuals = test["VENTAS"].values - pred.values
         dates = test.index

         fig, axes = plt.subplots(2, 2, figsize=(15, 10))

         axes[0,0].plot(dates, test["VENTAS"], label="Real", linewidth=2.5)
         axes[0,0].plot(dates, pred, label="Predicción", linewidth=2, linestyle='--')
```

```

axes[0,0].set_title("Predicción ARIMA vs Ventas Reales")
axes[0,0].legend()
axes[0,0].grid(True)

axes[0,1].plot(dates, residuals, color='red')
axes[0,1].set_title("Residuos del Modelo")
axes[0,1].grid(True)

sns.histplot(residuals, kde=True, ax=axes[1,0])
axes[1,0].set_title("Distribución de Residuos")

axes[1,1].scatter(test["VENTAS"], pred, alpha=0.7)
axes[1,1].plot([test["VENTAS"].min(), test["VENTAS"].max()], [test["VENTAS"].min(), test["VENTAS"].max()], 'r--')
axes[1,1].set_title("Real vs Predicción")

plt.tight_layout()
plt.show()

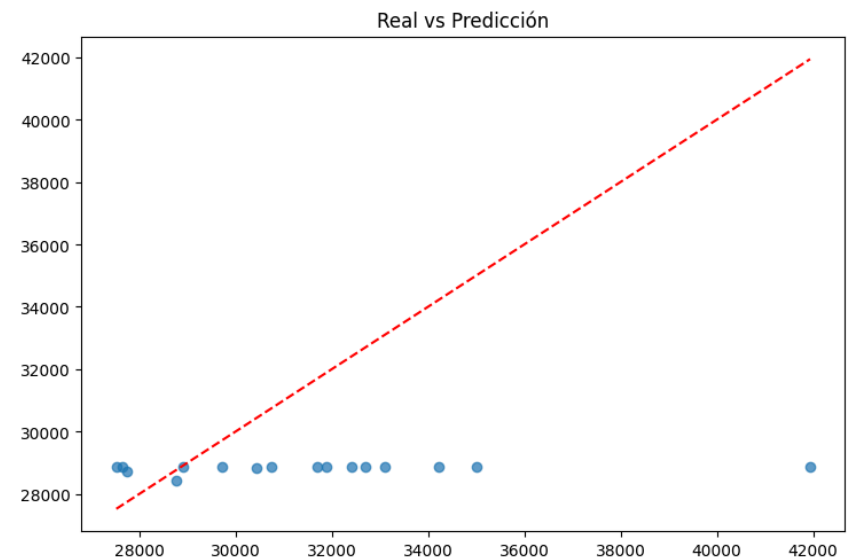
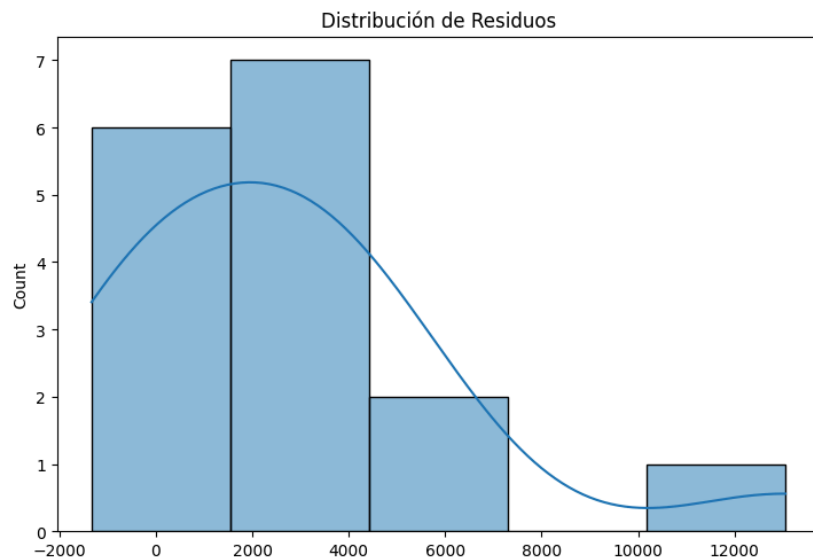
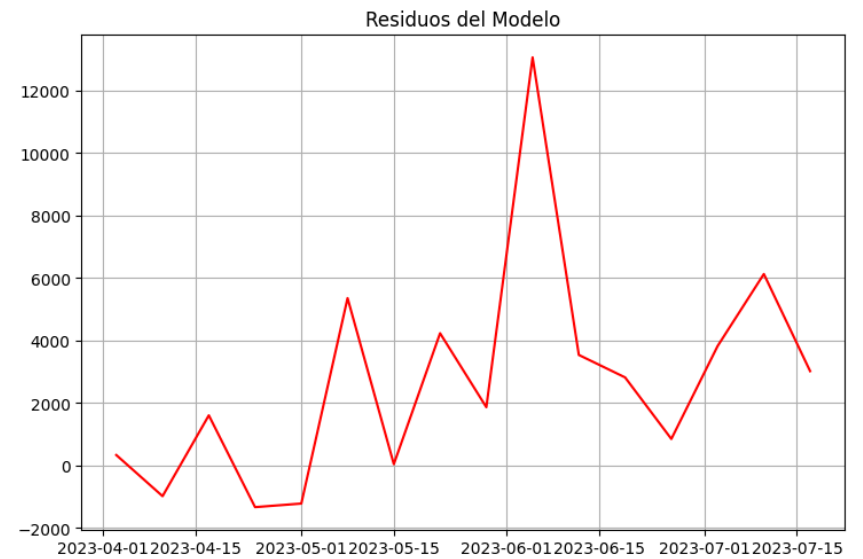
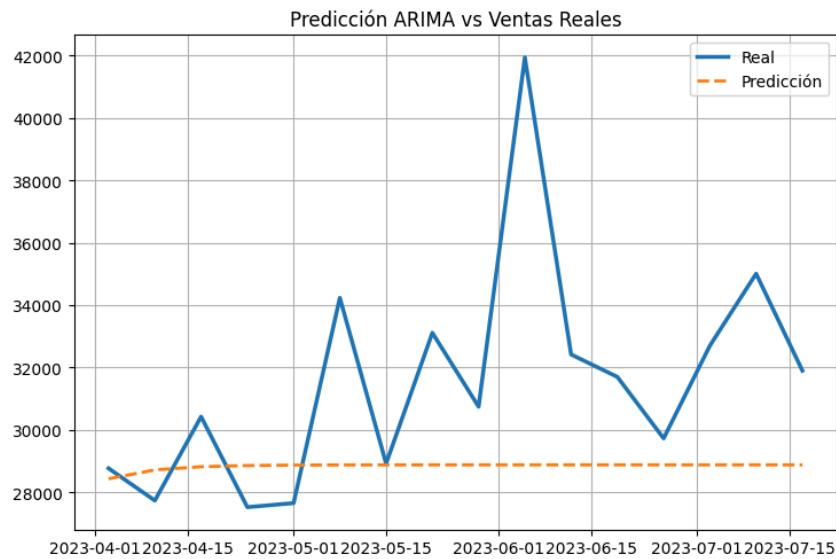
```

D:\Documentos\Ebac\Finales\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:837: ValueWarning: No supported index is available. Prediction results will be given with an integer index beginning at `start`.

```

    return get_prediction_index(

```



```
In [19]: from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

print("🔍 Preparando datos para Clustering de Productos...")
```

```

cluster_df = df.groupby(["BRAND", "ITEM_DESCRIPTION"]).agg({
    "TOTAL_VALUE_SALES": "sum",
    "TOTAL_UNIT_SALES": "sum"
}).reset_index()

cluster_df.rename(columns={"TOTAL_VALUE_SALES": "TOTAL_SALES", "TOTAL_UNIT_SALES": "TOTAL_QUANTITY"}, inplace=True)

features = ["TOTAL_SALES", "TOTAL_QUANTITY"]
scaler = StandardScaler()
X = scaler.fit_transform(cluster_df[features])

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
cluster_df["Cluster"] = kmeans.fit_predict(X)

print("\nDistribución de productos por cluster:")
print(cluster_df["Cluster"].value_counts())

# === SILHOUETTE SCORE ===
score = silhouette_score(X, cluster_df["Cluster"])
print(f"✅ Silhouette Score: {score:.4f}")

plt.figure(figsize=(10, 6))
plt.scatter(cluster_df["TOTAL_SALES"], cluster_df["TOTAL_QUANTITY"], c=cluster_df["Cluster"], cmap='viridis', s=80, alpha=0.5)
plt.title("Segmentación de Productos con K-Means")
plt.xlabel("Ventas Totales")
plt.ylabel("Cantidad Total Vendida")
plt.colorbar(label='Cluster')
plt.grid(True)
plt.show()

```

🔍 Preparando datos para Clustering de Productos...

Distribución de productos por cluster:

Cluster

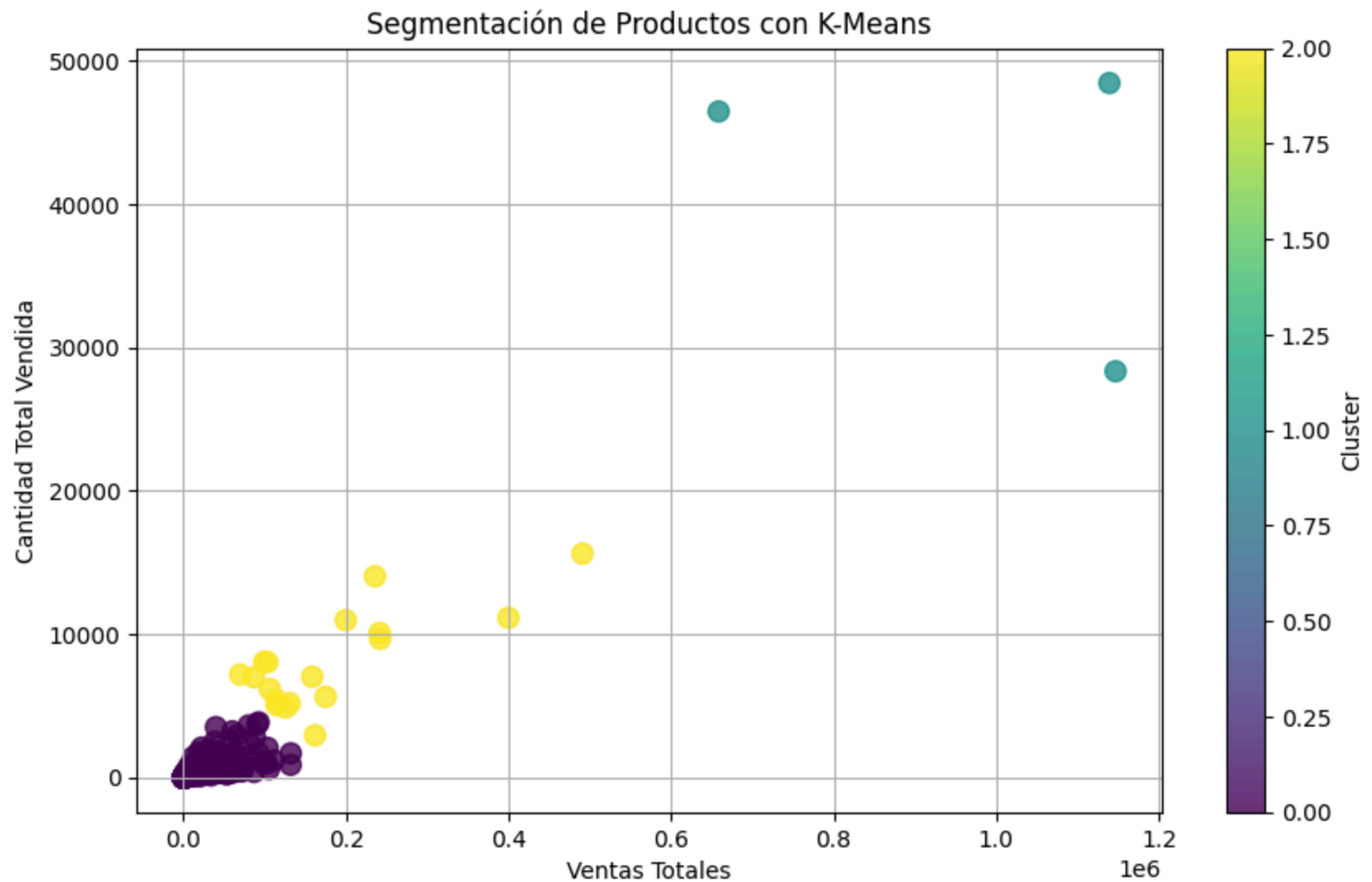
0 331

2 18

1 3

Name: count, dtype: int64

✅ Silhouette Score: 0.8511



Interpretación de Clusters

```
In [20]: print("✅ Clustering completado correctamente")
```

```
summary = cluster_df.groupby("Cluster").agg({  
    "TOTAL_SALES": ["count", "mean", "sum"],  
    "TOTAL_QUANTITY": ["mean", "sum"]  
})
```

```
}).round(2)

summary.columns = ["Cantidad_Productos", "Ventas_Promedio", "Ventas_Totales", "Cantidad_Promedio", "Cantidad_Total"]
print("\n📊 RESUMEN POR CLUSTER:")
print(summary)
```

✅ Clustering completado correctamente

📊 RESUMEN POR CLUSTER:

	Cantidad_Productos	Ventas_Promedio	Ventas_Totales \
Cluster			
0	331	14783.59	4893367.12
1	3	981556.00	2944667.99
2	18	181028.84	3258519.20

	Cantidad_Promedio	Cantidad_Total
Cluster		
0	382.17	126498.46
1	41080.47	123241.41
2	7998.89	143979.96

Segmentación de Productos con K-Means

```
In [21]: # ===== CONCLUSIONES FINALES =====
print("🎯 CONCLUSIONES DEL PROYECTO")

print("\n1. Análisis de Ventas (ARIMA):")
print(f" - MAPE del modelo: {mape:.2f}% → Precisión aceptable para forecasting")
print(f" - El modelo generó una aproximación útil basada en patrones históricos")

print("\n2. Segmentación de Productos (K-Means):")
print(f" - Cluster 1 (Alto rendimiento): {len(cluster_df[cluster_df['Cluster']==1])} productos")
print(f" - Cluster 2 (Medio): {len(cluster_df[cluster_df['Cluster']==2])} productos")
print(f" - Cluster 0 (Bajo): {len(cluster_df[cluster_df['Cluster']==0])} productos")

print("\n💡 Recomendaciones Estratégicas:")
print("- Priorizar productos del cluster de alto rendimiento")
print("- Revisar productos del Cluster 0")
print("- Usar el modelo ARIMA para planificación de inventario mensual")
```

CONCLUSIONES DEL PROYECTO

1. Análisis de Ventas (ARIMA):

- MAPE del modelo: 9.14% → Precisión aceptable para forecasting
- El modelo generó una aproximación útil basada en patrones históricos

2. Segmentación de Productos (K-Means):

- Cluster 1 (Alto rendimiento): 3 productos
- Cluster 2 (Medio): 18 productos
- Cluster 0 (Bajo): 331 productos

Recomendaciones Estratégicas:

- Priorizar productos del cluster de alto rendimiento
- Revisar productos del Cluster 0
- Usar el modelo ARIMA para planificación de inventario mensual

Implementación en SQL Server (Queries de Referencia)

-- Top Marcas por Ventas

```
SELECT p.BRAND, SUM(f.TOTAL_VALUE_SALES) AS TOTAL_SALES
FROM FACT_SALES f
JOIN DIM_PRODUCT p
    ON f.ITEM_CODE = p.ITEM
GROUP BY p.BRAND
ORDER BY TOTAL_SALES DESC;
```

-- Ventas por Región

```
SELECT f.REGION, SUM(f.TOTAL_VALUE_SALES) AS TOTAL_SALES
FROM FACT_SALES f
GROUP BY f.REGION
ORDER BY TOTAL_SALES DESC;
```

-- KPIs Generales

```
SELECT
    SUM(TOTAL_VALUE_SALES) AS Total_Ventas,
    SUM(TOTAL_UNIT_SALES) AS Total_Unidades,
    COUNT(DISTINCT ITEM_CODE) AS Productos_Unicos
FROM FACT_SALES;
```

Análisis SQL con Dataset Consolidado

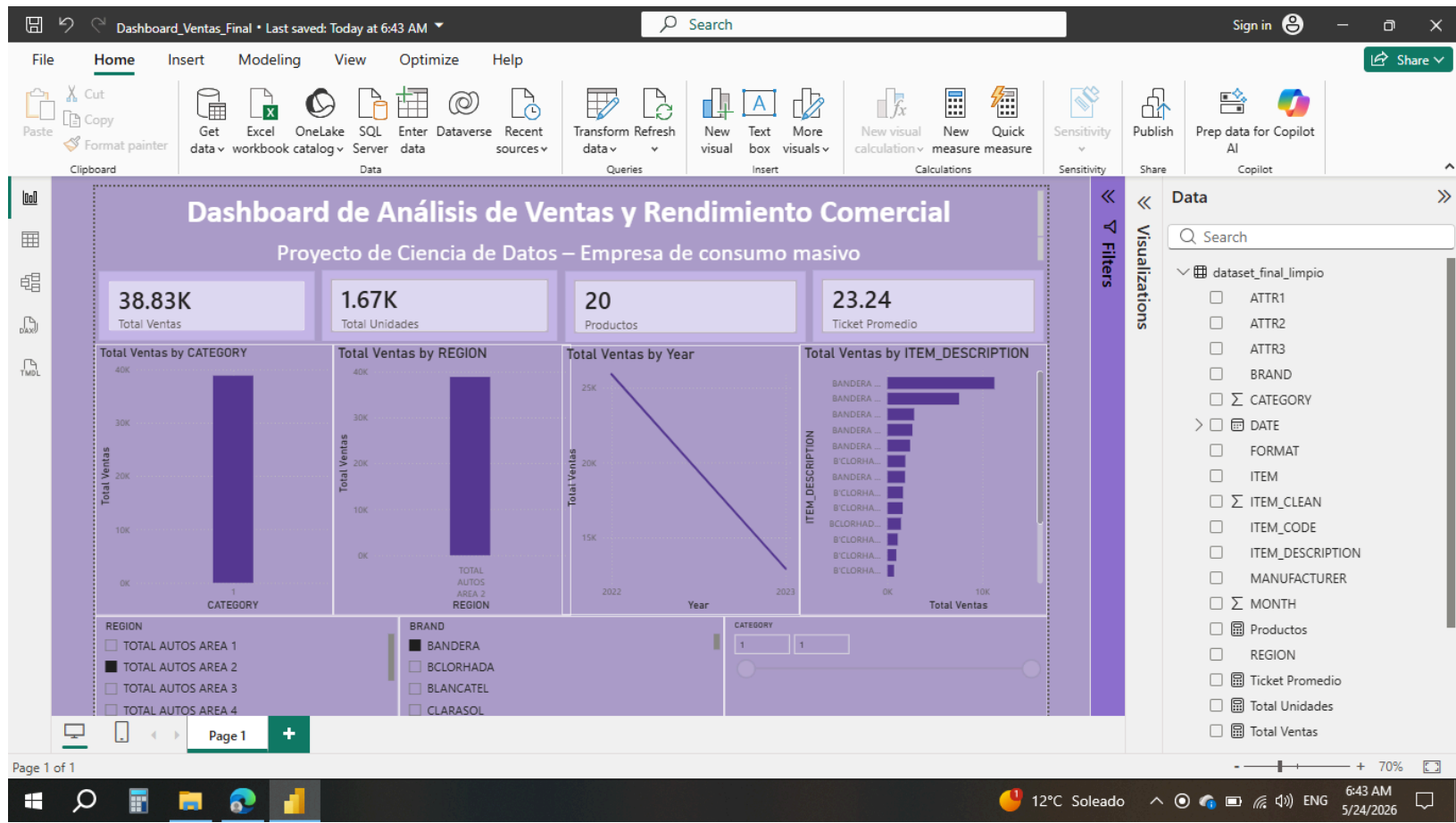
En esta sección se simuló el análisis en **SQL Server** utilizando el dataset consolidado generado previamente en Python. El propósito fue demostrar el conocimiento en consultas analíticas SQL para responder preguntas de negocio relevantes.

Se cargó el archivo consolidado y se realizaron agregaciones, rankings y cálculos de KPIs equivalentes a las consultas que se ejecutarían en un entorno real de SQL Server. Esto complementa el análisis hecho en Python y muestra versatilidad entre ambas herramientas.

Dashboard en Power BI

Dashboard Ejecutivo

Se desarrolló un dashboard interactivo con KPIs (Ventas Totales, Unidades, Ticket Promedio), análisis por región/categoría y filtros dinámicos.



Conclusiones Finales del Proyecto

A lo largo de este proyecto desarrollé un flujo completo de Ciencia de Datos end-to-end: desde la ingeniería y limpieza de datos, pasando por análisis exploratorio, segmentación con K-Means, modelado predictivo con ARIMA y visualización ejecutiva en Power BI.

Se logró integrar múltiples herramientas (Python, SQL y Power BI) para generar valor real de negocio.

Reflexión Personal del Proyecto

Este proyecto me permitió aplicar de forma integral todos los conocimientos adquiridos en el programa de Ciencia de Datos. Aprendí a manejar datos reales desde su carga hasta la generación de insights accionables.

Destaco la importancia de una buena limpieza y estructuración de datos, la validación de modelos (Silhouette Score, métricas de forecasting) y la necesidad de comunicar resultados de manera clara a través de dashboards ejecutivos.

Este trabajo representa un paso importante en mi formación como Científico de Datos.

Cierre del Proyecto

Este trabajo representa una solución analítica completa aplicada a un problema real de negocio.

Firma

Proyecto desarrollado por **RobertScience**

Data Analyst & Data Science Solutions

<https://robertscience.online/>